

Conway: grid indexing and next-state timing

Derive $\text{row} * 16 + \text{col}$, wrap both coordinates, count eight old-state neighbors and advance the complete board on one clock edge.

Kapil Tripathi | Verilog & digital design | 7 pages

CONTENTS / [click a row to jump](#)

Row/column indexing and inverse mapping	2
Toroidal boundaries and eight neighbors	3
Rules, old state and next-state timing	4
Complete Verilog solution	5
Loops, arithmetic width and storage	6
Boundary trace and revision checks	7

KEEP THIS IDEA

Calculate every next cell from the same old grid. Wrap row and column separately, and let load override evolution at the rising edge.

Tracker entries: 177

Study notes by Kapil Tripathi, based on HDLBits exercises. Original questions, source references and dated verification records are retained in the notes.

ENTRY 177 / GRID REPRESENTATION

Why the index is $\text{row} * 16 + \text{col}$

The circuit stores 256 cells in `q[255:0]`. A row has 16 cells, so the start of row r is bit $16r$. Column c selects the c -th bit after that start.

Your editor question: "why and how to calc the row and col index, discuss."

Derive the address from rows already skipped

Before row 3 there are three complete rows: $3 \times 16 = 48$ cells. Moving to column 5 adds five more positions. Therefore cell (3,5) is `q[53]`. Both coordinates start at zero. Multiplication by 16 supplies the row offset; addition supplies the position inside that row.

Coordinate or row	Vector location	Reason
(0,0)	<code>q[0]</code>	$0 \times 16 + 0$
Row 0	<code>q[15:0]</code>	Indices 0 through 15
Row 1	<code>q[31:16]</code>	Indices 16 through 31
(3,5)	<code>q[53]</code>	$48 + 5$
(15,15)	<code>q[255]</code>	$240 + 15$

Recover the coordinates

```
index = row * 16 + col;  
row   = index / 16; // integer quotient  
col   = index % 16; // remainder
```

For index 53, integer division gives row 3 and the remainder gives column 5. For a nonnegative 8-bit index, the upper four bits identify the row and the lower four identify the column. This is the binary meaning of dividing a 16-column grid into rows.

Vector print order versus coordinate order

The notation `q[15:0]` prints bit 15 on the left, but our coordinate convention calls `q[0]` column 0. Keep that convention consistent in neighbors and traces. Reversing how a row is drawn does not change the recurrence if every coordinate mapping is reversed consistently.

Source: [HDLBits Conwaylife](#). The problem specifies the row-to-vector mapping and a 16 x 16 grid.

BOUNDARIES / TOROIDAL NEIGHBORS

Wrap coordinates before flattening

A toroid connects top to bottom and left to right. Wrap the row and column independently, then turn each coordinate pair into a bit index.

```
up    = (row == 0) ? 15 : row - 1;
down  = (row == 15) ? 0 : row + 1;
left  = (col == 0) ? 15 : col - 1;
right = (col == 15) ? 0 : col + 1;
```

The eight neighbors of cell (0,0)

Direction	Wrapped coordinate	Read from old q
Up-left	(15,15)	q[255]
Up	(15,0)	q[240]
Up-right	(15,1)	q[241]
Left	(0,15)	q[15]
Right	(0,1)	q[1]
Down-left	(1,15)	q[31]
Down	(1,0)	q[16]
Down-right	(1,1)	q[17]

Why flat index + 1 can be wrong

At (0,15), the right neighbor is (0,0), index 0. Incrementing flat index 15 gives 16, which is (1,0), a different row. Even taking $(\text{index} + 1) \% 256$ does not fix that row-boundary error. Wrap col within 0..15 first, while preserving row.

Do not count the center

The surrounding 3 x 3 square contains nine positions. The sum uses its eight outer positions and excludes $q[\text{row} \times 16 + \text{col}]$. The center matters only for the two-neighbor survival rule. Every cell, including a corner, has eight distinct neighbors on this 16 x 16 toroid.

Source: [HDLBits Conwaylife](#). The listed corner neighbors require wrapping on both axes.

RULES / PRESENT AND NEXT VALUES

Every cell reads the same old generation

q is the registered current grid. $next_q$ is the combinational result of applying one generation of rules to that grid. All neighbor reads come from q .

Live neighbors	Current cell 0	Current cell 1	Assignment
0 or 1	0	0	next cell = 0
2	0	1	next cell = old cell
3	1	1	next cell = 1
4 through 8	0	0	next cell = 0

Why copy q when count is two?

Two neighbors preserve the center's existing state: a dead center stays dead and a live center stays live. Setting it unconditionally to 1 would incorrectly create a new cell. Copying $q[\text{row} \cdot 16 + \text{col}]$ expresses precisely that conditional survival.

A generation must use one consistent snapshot

Suppose an earlier loop iteration kills a neighbor. A later cell must still see that neighbor's old live value when computing this same generation. Reading partly updated $next_q$ would mix two generations and make the result depend on loop traversal order. Reading only q avoids this contamination.

```
// Between rising edges:
next_q = F(q);
```

```
// At the rising edge:
if (load) q <= data;
else      q <= next_q;
```

$next_q$ is a signal, not a function call

The combinational process already calculates $next_q$ as q changes. At the edge, $q \leq next_q$ samples that calculated value and schedules the register update. After q changes, the combinational logic settles to the following generation's candidate. There is still only one registered advance per clock.

When a separate next value is optional

A single clocked process can instead assign each q bit with a nonblocking expression that reads only old q . In that version no $next_q$ signal is needed. Keep the two-process form here because it clearly separates neighbor calculation from storage. Do not register $next_q$ in another clocked block unless an additional pipeline cycle is intended.

Source: [HDLBits Conwaylife](#). Rules and the one-generation-per-clock requirement.

COMPLETE RTL / ONE OWNER FOR q

The complete two-process solution

This is the editor solution with formatting and comments clarified. The combinational block assigns next_q; the rising-edge block alone assigns q.

```
module top_module (
    input clk,
    input load,
    input [255:0] data,
    output reg [255:0] q
);
    reg [255:0] next_q;
    integer row, col, up, down, left, right, count;

    always @(*) begin
        next_q = q;
        for (row = 0; row < 16; row = row + 1) begin
            for (col = 0; col < 16; col = col + 1) begin
                up    = (row == 0) ? 15 : row - 1;
                down  = (row == 15) ? 0  : row + 1;
                left  = (col == 0) ? 15 : col - 1;
                right = (col == 15) ? 0  : col + 1;
                count = q[up*16+left] + q[up*16+col]
                    + q[up*16+right] + q[row*16+left]
                    + q[row*16+right] + q[down*16+left]
                    + q[down*16+col] + q[down*16+right];
                if (count == 2)
                    next_q[row*16+col] = q[row*16+col];
                else if (count == 3)
                    next_q[row*16+col] = 1'b1;
                else
                    next_q[row*16+col] = 1'b0;
            end
        end
    end

    always @(posedge clk) begin
        if (load) q <= data;
        else     q <= next_q;
    end
endmodule
```

load has priority: the loading edge stores data unchanged. The first evolved generation is stored on the next rising edge with load low. There is no reset port in this interface; load supplies the initial grid.

EXECUTION / WIDTHS AND STORAGE

What the loops and assignments mean in hardware

Verilog source order describes how to calculate a result within a process. Clock edges determine when the stored grid changes. These are different kinds of sequencing.

The nested loops do not take 256 clocks

Each fixed-bound loop visits 16 coordinates during one evaluation of the combinational block. Together they describe the next-value logic for all 256 cells. Synthesis can expand these constant bounds into hardware; one cell is not processed per edge. A resource-shared multi-cycle design would need an explicit controller, address counter and different timing.

Use blocking assignments for the calculation

up, down, left and right must be available before the neighbor sum is evaluated. count must be available before the if statement reads it. Blocking = gives that immediate procedural calculation order. Nonblocking <= is used for the clocked q update so the old grid is sampled before the new grid becomes visible.

Why integer count does not overflow at eight

An integer is a 32-bit signed variable in Verilog. In this assignment, the addition is context-sized by that 32-bit destination, so the eight one-bit values are added at a sufficient width. All known inputs are 0 or 1, giving a result from 0 through 8. This code does not truncate each pairwise sum to one bit.

If refactoring the count into a function, concatenation or differently sized intermediate, check that expression's sizing rules again. Four unsigned bits are enough to represent 0..8. Explicitly zero-extending each bit to four bits before addition is a clear portable way to make the required width visible.

```
// Example of explicit width for one summand:
{3'b000, q[up*16+left]}
```

Defaults, latches and temporary variables

next_q = q provides a complete default before the loop. In this exact code, all 256 bits also receive an assignment through the complete if/else chain, so the default is redundant but safe. The count==2 branch still explicitly implements hold. Removing both default coverage and an assignment path would risk a latch in a combinational process.

The integer declarations do not themselves create flip-flops. Here the loop coordinates and count are procedural calculation temporaries. q is the clocked storage. The synthesized circuit still needs the logic for all cell results; using one source-level count variable does not promise one time-shared physical adder.

Connect this to the timer-series question

state <= next_state and q <= next_q both store a precomputed next value. In a timer, a test of state asks what state we are leaving; a test of next_state asks the chosen destination. Conway has no named controller states: q and next_q are the present and next complete board configurations. Neither name is a function invocation.

WORKED TRACE / REVISION CHECKS

Trace the boundary, then test the whole grid

The seed 256'h7 lights row 0, columns 0, 1 and 2. It is a useful boundary test because its next generation reaches row 15 across the top seam.

Stored generation	Live coordinates	Set bit indices
After load	(0,0), (0,1), (0,2)	0, 1, 2
After one advance	(15,1), (0,1), (1,1)	241, 1, 17
After two advances	(0,0), (0,1), (0,2)	0, 1, 2

Explain the first transition cell by cell

The center (0,1) sees two live neighbors and stays alive. The endpoints (0,0) and (0,2) each see only one and die. The dead cells directly above and below the center each see all three original live cells, so they are born. Above row 0 means row 15, giving the seam-crossing vertical line.

A loading edge is not an evolution edge

If load=1 and data=256'h7, that edge must leave q equal to 7, not the vertical line. While load remains high, each edge reloads the current data input. Once load becomes 0, each subsequent rising edge stores one generation. In simulation, inspect q after the nonblocking update has taken effect.

Useful independent checks

Case	Expected behavior / defect exposed
All zero	Stays zero; no spontaneous births
All one	Every cell has eight neighbors, so the next grid is zero
Single live cell	Dies after one advance
2 x 2 block	Each live cell has three neighbors; the block remains fixed
256'h7 blinker	Two-cycle oscillation across the top seam
Random loaded grids	Compare every output bit to an independently indexed model
All 512 local configurations	Both center values for all 256 eight-neighbor patterns

Final mental check

For any bit, recover row and col, wrap each neighbor coordinate, count eight old-q bits, apply the rule, and store the result on one edge. If a result differs, check coordinate wrapping before changing the rule. If the result is one generation late, check whether next_q was accidentally registered.

Source: [HDLBits Conwaylife](#). The seed-7 oscillator is the problem's boundary-test hint.